

DBMS

Database

- Database is a collection of related data and data is a collection of facts and figures that can be processed to produce information.
- Mostly data represents recordable facts.
- Data aids in producing information, which is based on facts.



Database Management System

- Database Management System or DBMS in short refers to the technology of storing and retrieving users' data with utmost efficiency along with appropriate security measures.
- Provide an efficient and easy way to store, update, and retrieve data from a database.
- Manage information about users who interact with the DBMS, and the tasks that these users can perform on the data.
- Providing methods for data manipulation, such as adding, editing, and deleting data.

Users

- A typical DBMS has users with different rights and permissions who use it for different purposes.
- Some users retrieve data and some back it up. The users of a DBMS can be broadly categorized as follows:



- The most popular data model in DBMS is the Relational Model. It is more scientific model than others.

The main highlights of this model are:

- Data is stored in tables called relations.
- Relations can be normalized.
- In normalized relations, values saved are atomic values.
- Each row in a relation contains a unique value.

DBMS Used For

- Data independence and efficient access.
- Reduced application development time.
- Data integrity and security.
- Uniform data administration.
- Concurrent access, recovery from crashes.

Need For DBMS

- 1) Processing Queries and Object Management.
- 2) Controlling redundancy and inconsistency.
- 3) Efficient memory management and indexing.
- 4) Concurrency control and transaction management.
- 5) Access Control and ease in accessing data.
- 6) Integrity constraints.

Data models

Hierarchical Model

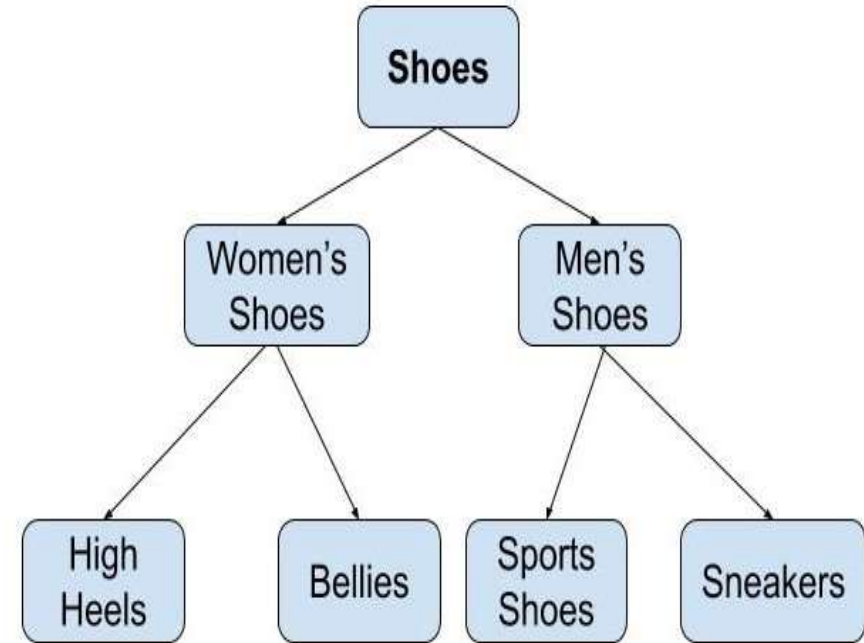
The hierarchy starts from the root which has root data and then it expands in the form of a tree adding child node to the parent node.

1. *One-to-many relationship:* .

2. *Parent-Child Relationship*

3. *Deletion Problem:*

4. *Pointers:*



Hierarchical Model

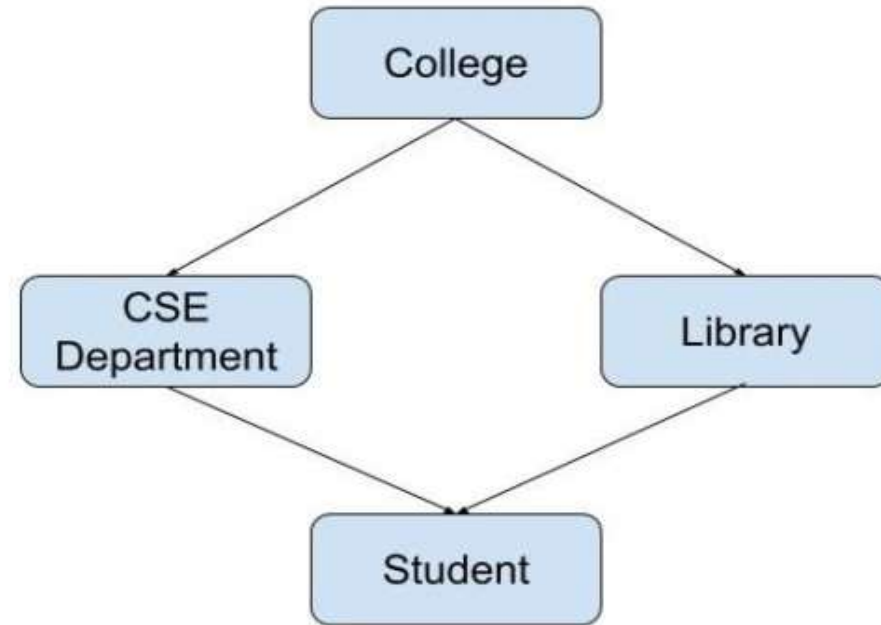
Network Model

A record can have more than one parent. It replaces the hierarchical tree with a graph.

1. Ability to Merge more Relationships:

2. Many paths:

3. Circular Linked List:



Network Model

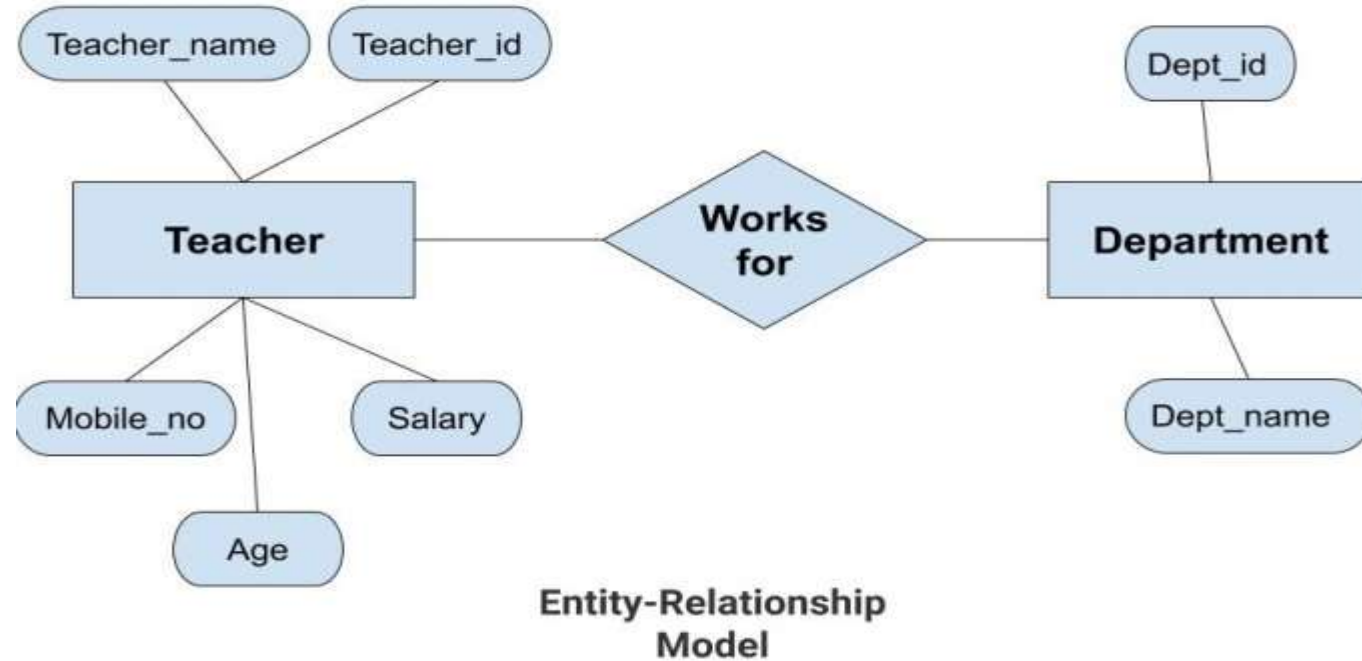
Entity-Relationship Model

Entity-Relationship Model or simply ER Model is a high-level data model diagram

- *Graphical Representation for*

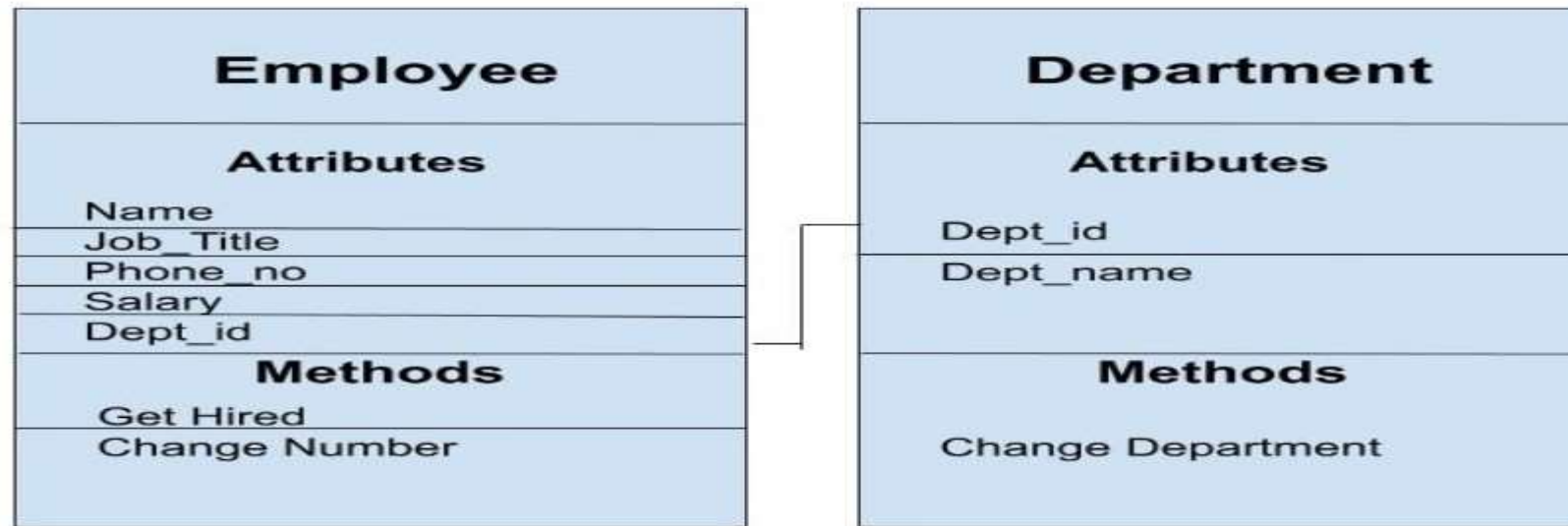
Better Understanding:

- *ER Diagram:*
- *Database Design:*



Object-Oriented Model

- Object DBMSs add database functionality to object programming languages. They bring much more than persistent storage of programming language objects.



Object_Oriented_Model

Relational Model

- (RDBMS - relational database management system) A database based on the relational model developed by E.F. Code.
- In such a database the data and relations between them are organized in tables. A table is a collection of records and each record in a table contains the same fields.

Emp_id	Emp_name	Job_name	Salary	Mobile_no	Dep_id	Project_id
AfterA001	John	Engineer	100000	9111037890	2	99
AfterA002	Adam	Analyst	50000	9587569214	3	100
AfterA003	Kande	Manager	890000	7895212355	2	65

EMPLOYEE TABLE

RDBMS

- A Relational Database management System(RDBMS) is a database management system based on relational data model introduced by E.F Codd.
- Relational data model is the primary data model, which is used widely around the world for data storage and processing.
- This model is simple and it has all the properties and capabilities required to process data with storage efficiency.
- RDBMS is the basis for SQL and for all modern database systems like MS SQLServer, IBM DB2, Oracle, MySQL, and Microsoft Access.

- Relational database is a collection of organized set of tables from which data can be accessed easily.
- An RDBMS can be defined as a DBMS where all the data is organized strictly as tables of data values.
- Each table in a database has a unique table name.
- A record, also called a row of data, is each individual entry that exists in a table

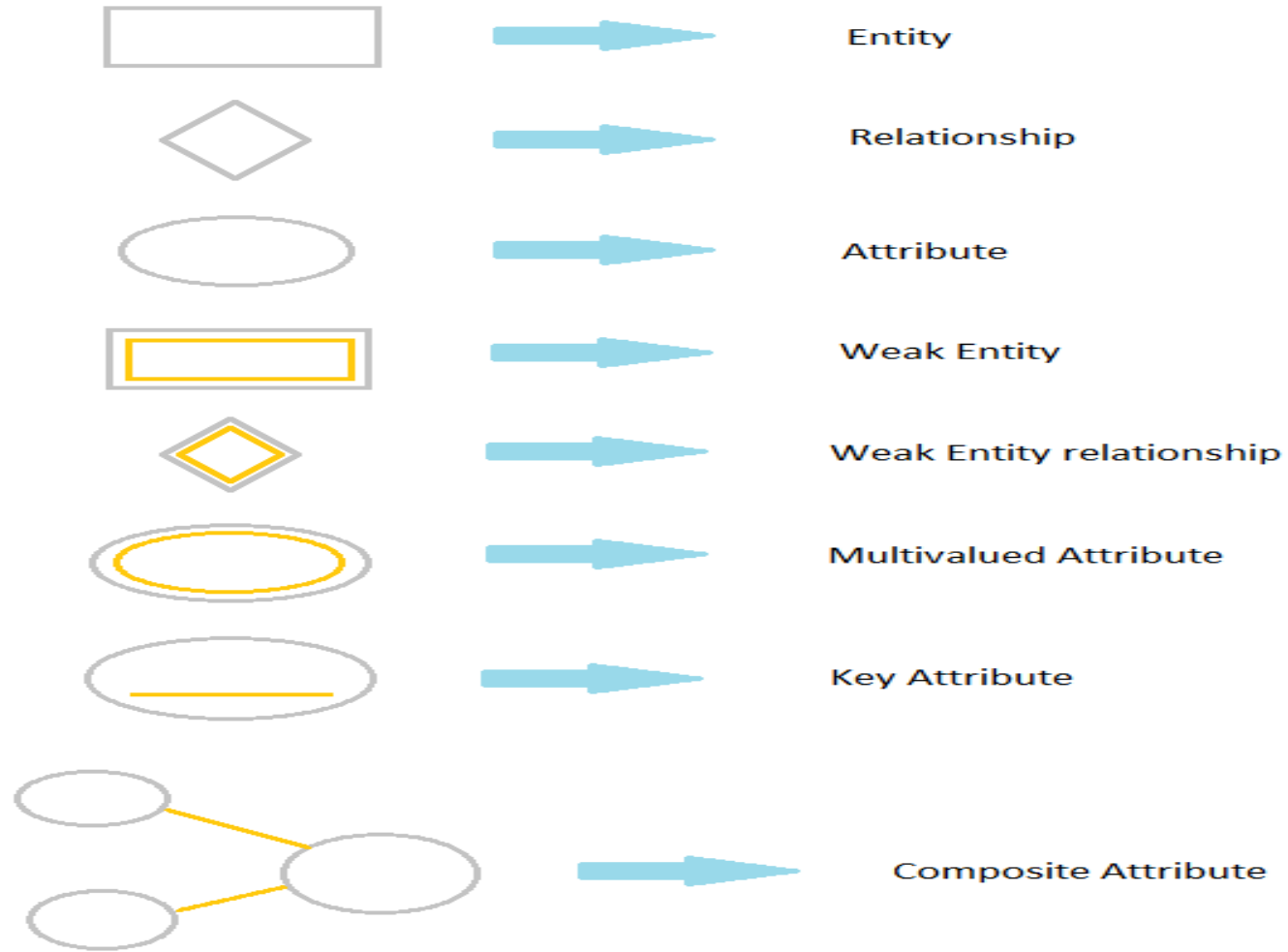
- The row (or record) in the table is called a tuple.
- The column (or field) is called an attribute.
- Every column in a table must have a unique name.
- The number of tuples is called the cardinality of the table.
- The number of attributes is called the degree of the table.
- A single entry in a table is called a Record or Row.
- A Record in a table represents set of related data

ER Diagram

- ER-Diagram is a visual representation of data that describes how data is related to each other.



Symbols and Notations



Components of E-R Diagram

The E-R diagram has three main components.

1)Entity

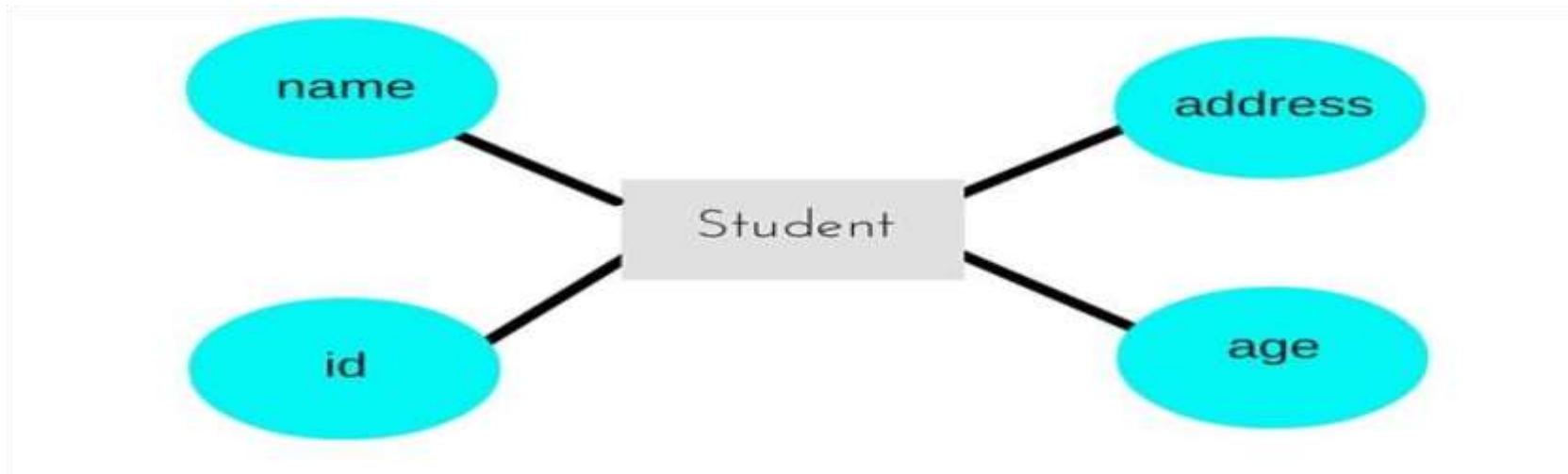
- An Entity can be any object,place, person or class.
- In E-R Diagram, an entity is represented using rectangles.

eg:- An Organisation . Employee, Manager, Department, Product and many more can be taken as entities from an Organisation.



2) *Attribute*

- An Attribute describes a property or characteristic of an entity.
- An attribute is represented using eclipse.
- Eg: Name, Age, Address etc can be attributes of a Student.



3) *Relationship*

A Relationship describes relations between entities. Relationship is represented using diamonds.

There are three types of relationship that exist between Entities.

1. Binary Relationship
2. RecursiveRelationship
3. TernaryRelationship



Binary Relationship

- Binary Relationship means relation between two Entities. This is further divided into three types.
- **One to One** : This type of relationship is rarely seen in real world.



- **One to Many** : It reflects business rule that one entity is associated with many number of same entity.



- **Many to One** : It reflects business rule that many entities can be associated with just one entity.

For example, Studenten rolls for only one Course but a Course can have many Students.



- *Many to Many :*



- The above diagram represents that many students can enroll for more than one courses.

Recursive Relationship

When an Entity is related with itself it is known as Recursive Relationship.



Properties of Relational Tables

- Values are atomic.
- All of the values in a column have the same data type.
- Each row is unique.
- The sequence of columns is insignificant.
- The sequence of rows is insignificant.
- Each column has a unique name.
- Integrity constraints maintain data consistency across multiple tables.

Advantages of DBMS

1. Data Integrity
2. Data Security
3. Better data integration
4. Minimized Data Inconsistency
5. Faster Data Access
6. Recovery and Backup
7. Increased end-user productivity
8. Simplicity

Disadvantages of DBMS

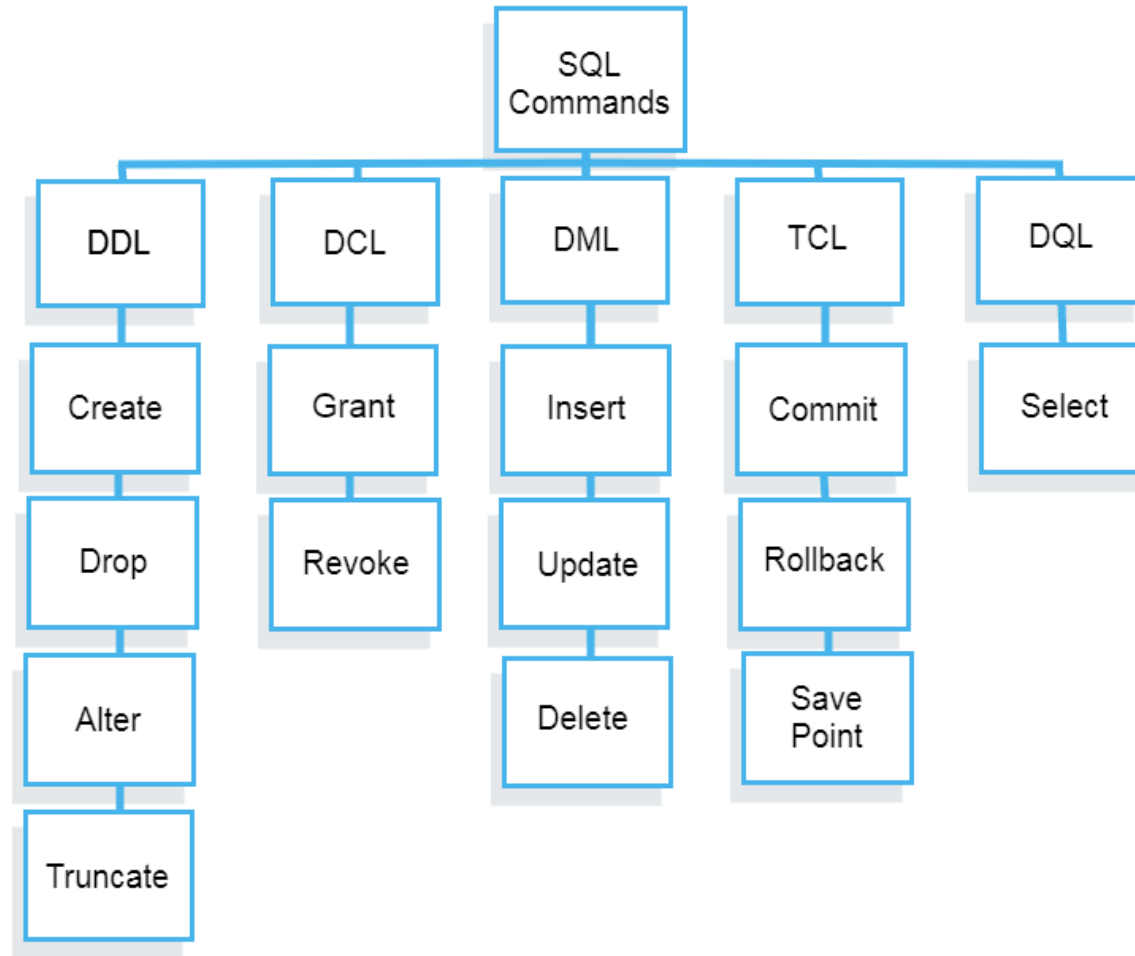
- Cost of Hardware and Software
- Cost of Data Conversion
- Cost of Staff Training
- Appointing Technical Staff
- Database Damage

SQL

- **SQL** is ***Structured Query Language***, which is a computer language for storing, manipulating and retrieving data stored in relational database.
- SQL is the standard language for Relation Database System.
- All relational database management systems like MySQL, MS Access, Oracle, Sybase, Informix, postgres and SQL Server use SQL as standard database language.

SQL Commands:

- The standard SQL commands to interact with relational databases are **CREATE, SELECT, INSERT, UPDATE, DELETE** and **DROP**.
- These commands can be classified into groups based on their nature:



DATA TYPES

- **Numeric data types** : such as int, tinyint, bigint, float, real, etc.
- **Date and Time data types** : such as Date, Time, Datetime, etc.
- **Character and String data types** : such as char, varchar, text, etc.
- **Unicode character string data types**, for example nchar, nvarchar, ntext, etc.
- **Binary data types** such as binary, varbinary, etc.

DDL -Data Definition Language

- SQL uses the following set of commands to define the structure of database.

Command	Description
CREATE	Creates a new table, a view of a table, or other object in database
ALTER	Modifies an existing database object, such as a table.
DROP	Deletes an entire table, a view of a table or other object in the database.
TRUNCATE	Remove all of the rows from a table, regardless of whether or not any conditions are met and resets the table definition

DDL -Data Definition Language

- The CREATE statement is used to create a database or a table.

Syntax

CREATE DATABASE dbname;

CREATE TABLE table_name
(
column_name1 data_type(size),
column_name2 data_type(size),
column_name3 data_type(size),
....
);

DDL -Data Definition Language

Eg:-

```
CREATE DATABASE mydb;
```

Eg:-

```
CREATE TABLE Persons  
(  
    PersonID int(150),  
    LastName varchar(255),  
    FirstName varchar(255),  
    Address text,  
    City varchar(255)  
);
```

DDL -Data Definition Language

DROP

- Drops commands, views, tables, and databases from RDBMS.

Syntax

Drop database databasename;

Drop table tablename;

DDL -Data Definition Language

ALTER

- Modifies database schema.

Syntax:

ALTER TABLE table_name {ADD|DROP|MODIFY} column_name {data_type};

ALTER TABLE table_name RENAME TO new_table_name;

Eg:-

Alter table article add subject varchar;

Alter table studentdetails rename column ph to phonenum;

DDL -Data Definition Language

Truncate

The TRUNCATE command is used to remove all of the rows from a table, regardless of whether or not any conditions are met and resets the table definition.

Syntax

```
TRUNCATE TABLE table_name;
```


DML -Data Manipulation Language

- SQL is equipped with data manipulation language (DML).
- DML modifies the database instance by inserting, updating, and deleting its data.

DML -Data Manipulation Language

Command	Description
INSERT	Creates a record
UPDATE	Modifies records
DELETE	Deletes records

DML -Data Manipulation Language

SQL INSERT Query

- The SQL **INSERT INTO** Statement is used to add new rows of data to a table in the database.

Syntax:

INSERT INTO TABLE_NAME (column1, column2, column3,...columnN)] VALUES (value1, value2, value3,...valueN);

INSERT INTO TABLE_NAME VALUES (value1,value2,value3,...valueN);

DML -Data Manipulation Language

SQL SELECT Query

- Used to fetch the data from a database table which returns data in the form of result table (called result-sets).

Syntax:

SELECT column1, column2, columnN FROM table_name;

SELECT column1, column2, columnN FROM table_name WHERE [condition]

DML -Data Manipulation Language

SQL WHERE Clause

- The SQL **WHERE** clause is used to specify a condition while fetching the data from single table or joining with multiple tables.
- If the given condition is satisfied, then only it returns specific value from the table.

DML -Data Manipulation Language

SQL UPDATE Query

- The SQL **UPDATE** Query is used to modify the existing records in a table.

Syntax:

*UPDATE table_name SET column1 = value1, column2 = value2...., columnN = valueN
WHERE [condition];*

DML -Data Manipulation Language

SQL DELETE Query

- The SQL **DELETE** Query is used to delete the existing records from a table.

Syntax:

DELETE FROM table_name WHERE [condition];

Eg:-

DELETE FROM CUSTOMERS WHERE ID = 6;

DCL - Data Control Language

- DCL includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system.

Command	Description
GRANT	Gives a privilege to user
REVOKE	Takes back privileges granted from user

syntax of grant : **GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_USER;**

syntax of revoke : **REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;**

TCL -Transaction Control Language

TCL commands are used to control the execution of a transaction

- **COMMIT**: Commits a Transaction.

Syntax: **COMMIT;**

- **ROLLBACK**: Rollbacks a transaction in case of any error occurs.

Syntax: **ROLLBACK;**

- **SAVEPOINT**: Sets a save point within a transaction.

Syntax: **SAVEPOINT SAVEPOINT_NAME;**

DQL - Data Query Language

- The purpose of the DQL Command is to get some schema relation based on the query passed to it.
- We can define DQL as follows it is a component of SQL statement that allows getting data from the database and imposing order upon it
- List of DQL:

SELECT: It is used to retrieve data from the database.

syntax : **SELECT * FROM Table_name;**

SQL Constraints

- Constraints are the rules enforced on data columns on table.
- These are used to limit the type of data that can go into a table.
- This ensures the accuracy and reliability of the data in the database.

NOT NULL - Ensures that a column cannot have a NULL value

UNIQUE - Ensures that all values in a column are different

CHECK - Ensures that the values in a column satisfies a specific condition

DEFAULT - Sets a default value for a column if no value is specified

CREATE INDEX - Used to create and retrieve data from the database very quickly

NOT NULL Constraint:

- By default, a column can hold NULL values.
- If you do not want a column to have a NULL value, then you need to define such constraint on this
- column specifying that NULL is now not allowed for that column.
- **CREATE TABLE CUSTOMERS(ID INT NOT NULL,NAME VARCHAR (20) NOT NULL);**

UNIQUE Constraint:

- The UNIQUE Constraint prevents two records from having identical values in a particular column.
- **CREATE TABLE CUSTOMERS(ID INT NOT NULL,NAME VARCHAR (20) NOT NULL, AGE INT NOT NULL UNIQUE, ADDRESS VARCHAR (25));**

DEFAULT Constraint:

- SQL sets a DEFAULT value for a column when the table is created:
- **CREATE TABLE Persons (ID int NOT NULL,FirstName varchar(255), City varchar(255) DEFAULT 'Sandnes');**

SQL INDEX Keyword

CREATE INDEX

The **CREATE INDEX** command is used to create indexes in tables (allows duplicate values).

syntax :

```
CREATE INDEX index_name ON table_name (column_name);
```

DROP INDEX

The **DROP INDEX** command is used to delete an index in a table.

syntax :

```
ALTER TABLE table_name
```

```
DROP INDEX index_name;
```

DBMS Keys

KEYS in DBMS is an attribute or set of attributes which helps you to identify a row(tuple) in a relation(table). They allow you to find the relation between two tables

Why we need a Key?

- Keys help you to identify any row of data in a table.
- Allows you to establish a relationship between and identify the relation between tables
- Help you to enforce identity and integrity in the relationship.

DBMS Keys

- **Super Key** – A super key is a group of single or multiple keys which identifies rows in a table.
- **Primary Key** – is a column or group of columns in a table that uniquely identify every row in that table.
- **Candidate Key** – is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes.

DBMS Keys

- **Alternate Key** – is a column or group of columns in a table that uniquely identify every row in that table.
- **Foreign Key** – is a column that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity.

PRIMARY Key:

- A primary key is a field in a table which uniquely identifies each row/record in a database table.
- Primary keys must contain unique values. A primary key column cannot have NULL values.
- A table can have only one primary key, which may consist of single or multiple fields.
- When multiple fields are used as a primary key, they are called a composite key.
- If a table has a primary key defined on any field(s), then you can not have two records having the same value of that field(s).

```
CREATE TABLE CUSTOMERS(ID INT NOT NULL,NAME VARCHAR (20) NOT NULL,  
AGE INT NOT NULL, PRIMARY KEY (ID) );
```

Auto Increment

- Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table.
- Often this is the primary key field that we would like to be created automatically every time a new record is inserted.
- Syntax :

```
CREATE TABLE Table_name (  
    Column1 INT NOT NULL AUTO_INCREMENT,  
    Column2 varchar(255),  
    PRIMARY KEY (Column1));
```

FOREIGN Key:

- A foreign key is a key used to link two tables together. This is sometimes called a referencing key.
- Foreign Key is a column or a combination of columns whose values match a Primary Key in a different table.
- The relationship between 2 tables matches the Primary Key in one of the tables with a Foreign Key in the second table.

```
CREATE TABLE ORDERS (  
    ID INT NOT NULL,  
    DATE DATETIME,  
    CUSTOMER_ID INT ,  
    AMOUNT double,  
    PRIMARY KEY (ID) ,  
    Foreign Key(CUSTOMER_ID ) references CUSTOMERS(ID),  
);
```

Operators

- An operator is a reserved word or a character used primarily in an SQL statement's WHERE
- clause to perform operation(s), such as comparisons and arithmetic operations.
- Operators are used to specify conditions in an SQL statement and to serve as conjunctions for
- multiple conditions in a statement.
 - Arithmetic operators
 - Comparison operators
 - Logical operators

AND and OR Operators

- The SQL AND and OR operators are used to combine multiple conditions to narrow data in an SQL statement.
- These two operators are called conjunctive operators.
- These operators provide a means to make multiple comparisons with different operators in the same SQL statement.

The AND Operator:

- The AND operator allows the existence of multiple conditions in an SQL statement's WHERE clause.

Syntax: The basic syntax of AND operator with WHERE clause is as follows:

```
SELECT column1, column2, columnN  
FROM table_name  
WHERE [condition1] AND [condition2]...AND [conditionN];
```

Eg:- **SELECT ID, NAME, SALARY**

FROM CUSTOMERS WHERE SALARY > 2000 AND age < 25;

The OR Operator:

- The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.

Syntax: The basic syntax of OR operator with WHERE clause is as follows:

```
SELECT column1, column2, columnN  
FROM table_name  
WHERE [condition1] OR [condition2]...OR [conditionN]
```

- Eg:-

```
SELECT ID, NAME, SALARY  
FROM CUSTOMERS  
WHERE SALARY > 2000 OR age < 25;
```

SQL LIKE Clause

- The SQL LIKE clause is used to compare a value to similar values using wildcard operators. There are
- two wildcards used in conjunction with the LIKE operator:
 - The percent sign (%)
 - The underscore (_)
 - The percent sign represents zero, one, or multiple characters.
 - The underscore represents a single number or character. The symbols can be used in combinations.

- Syntax: The basic syntax of % and _ is as follows:

```
SELECT FROM table_name  
WHERE column LIKE 'XXXX%'
```

or

```
SELECT FROM table_name  
WHERE column LIKE '%XXXX%'
```

or

```
SELECT FROM table_name  
WHERE column LIKE 'XXXX_'
```

or

```
SELECT FROM table_name  
WHERE column LIKE '_XXXX'
```

or

```
SELECT FROM table_name  
WHERE column LIKE '_XXXX_'
```

BETWEEN Operator

- The BETWEEN operator is used to search for values that are within a set of values, given the minimum value and the maximum value.
- Syntax

```
SELECT COLOUMN_NAME FROM CUSTOMERS  
WHERE AGE BETWEEN minimum_value AND maximum_value;
```

- Eg:

```
SELECT * FROM CUSTOMERS WHERE AGE BETWEEN 25 AND 27;
```

IN operator

- The IN operator is used to compare a value to a list of literal values that have been specified.

- Syntax

```
SELECT COLOUMN_NAME FROM CUSTOMERS  
WHERE AGE IN ( LITERAL1, LITERAL2 );
```

- Eg:

```
SELECT * FROM CUSTOMERS WHERE AGE IN ( 25, 27 );
```

SQL Alias

- You can rename a table or a column temporarily by giving another name known as alias.
- The use of table aliases means to rename a table in a particular SQL statement.
- The renaming is a temporary change and the actual table name does not change in the database.
- The column aliases are used to rename a table's columns for the purpose of a particular SQL query.

Syntax: The basic syntax of table alias is as follows:

```
SELECT column1, column2....  
FROM table_name AS alias_name  
WHERE [condition];
```

- The basic syntax of column alias is as follows:

```
SELECT column_name AS alias_name  
FROM table_name  
WHERE [condition];
```

Eg:

```
SELECT C.ID, C.NAME, C.AGE, O.AMOUNT  
FROM CUSTOMERS AS C, ORDERS AS O  
WHERE C.ID = O.CUSTOMER_ID;
```

SQL LIMIT Clause

- To limit the number of rows returned by a select statement, you use the LIMIT and OFFSET clauses.
- The following shows the syntax of LIMIT & OFFSET clauses:

SELECT column_list FROM table1
ORDER BY column_list
LIMIT row_count OFFSET offset;

eg 1 : **SELECT employee_id, first_name, last_name**
FROM employees ORDER BY first_name
LIMIT 5;

eg 2 : **SELECT employee_id, first_name, last_name**
FROM employees ORDER BY first_name
LIMIT 5 OFFSET 3;

Aggregate Functions [and performance scaling]

Aggregate functions are those functions in the DBMS which takes the values of multiple rows of a single column and then form a single value by using a query.

Following are the five aggregate functions:

1. AVG
2. COUNT
3. SUM
4. MIN
5. MAX

Aggregate Functions [and performance scaling]

AVG Function

- This function takes the values from the given column and then returns the average of the values

Eg:

Select AVG (Emp_salary) from Employee_Details ;

Aggregate Functions [and performance scaling]

COUNT Function

- This aggregate function returns the total number of values in the specified column.

Eg:

Select COUNT (Emp_salary) from Employee_Details ;

Aggregate Functions [and performance scaling]

SUM Function

- This aggregate function sums all the non-NULL values of the given column.

Eg

Select SUM (Emp_salary) from Employee_Details ;

Aggregate Functions [and performance scaling]

MAX Function

- This function returns the value, which is maximum from the specified column.

Eg

Select MAX (Emp_salary) from Employee_Details ;

Aggregate Functions [and performance scaling]

MIN Function

- This function returns the value, which is minimum from the specified column.

Eg:

Select MIN (Emp_salary) from Employee_Details ;

Aggregate Enhancers [order by/ group by/ having]

The use of ORDER BY/ GROUP BY are closely related to the use of aggregate functions and the HAVING statement.

- The ORDER BY clause's purpose is to sort the query result by specific columns.
- The GROUP BY clause's purpose is summarize unique combinations of columns values.

Aggregate Enhancers [order by/ group by/ having]

ORDER BY Clause

- ORDER BY clause allows sorting the output values.
- By default, the ascending (ASC) sorting is applied. For the sorting to be descending, an additional DESC clause is used.

```
SELECT column1, column2, columnN FROM table name  
ORDER BY column1, column2, columnN ASC | DESC;
```

Aggregate Enhancers [order by/ group by/ having]

GROUP BY Clause

- GROUP BY clause used to combine the selection results by one or more columns.
- After grouping, there will be only one entry for each value used in the column.

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Aggregate Enhancers [order by/ group by/ having]

HAVING Clause

- HAVING is a filtering tool. It indicates the result of aggregate functions performance.
- The HAVING clause is used when we cannot use the WHERE keyword, i.e., with aggregate functions.

.

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
```

MySQL Functions

MySQL has many built-in functions.

This reference contains string, numeric, date, and some advanced functions in MySQL.

SOME FUNCTIONS ARE:

BIN - Returns a binary representation of a number

DATABASE - Returns the name of the current database

ABS -Returns the absolute value of a number

CEIL -Returns the smallest integer value that is $\geq$ to a number

String Functions

ASCII

Returns the ASCII value for the specific character

CHAR_LENGTH

Returns the length of a string (in characters)

FORMAT

Formats a number to a format like "#,###,###.##", rounded to a specified number of decimal places

LCASE

Converts a string to lower-case

LEFT

Extracts a number of characters from a string (starting from left)

LENGTH

Returns the length of a string (in bytes)

LOCATE

Returns the position of the first occurrence of a substring in a

string

REPLACE

Replaces all occurrences of a substring within a string, with a new

substring

REVERSE

Reverses a string and returns the result

Date Functions

DATE_FORMAT

Formats a date

CURRENT_DATE

Returns the current date

DAY

Returns the day of the month for a given date

DAYOFYEAR

Returns the day of the year for a given date

DAYOFMONTH

Returns the day of the month for a given date

HOUR

Returns the hour part for a given date

LOCALTIME

Returns the current date and time

SUBDATE

Subtracts a time/date interval from a date and then returns the date

SQL Subqueries

A subquery is a SQL query nested inside a larger query.

A subquery may occur in :

- A SELECT clause
- A FROM clause
- A WHERE clause

You can use the comparison operators, such as >, <, or =. The comparison operator can also be a multiple-row operator, such as IN, ANY, or ALL.

syntax :

```
SELECT  select_list
FROM    table
WHERE   expr operator
        (SELECT  select_list
         FROM    table);
```

Handling Duplicates

- The SQL **DISTINCT** keyword is used in conjunction with SELECT statement to eliminate all the duplicate records and fetching only unique records.

Syntax:

*SELECT DISTINCT column1, column2,.....columnN FROM table_name WHERE
[condition]*

EG:

SELECT DISTINCT SALARY FROM CUSTOMERS ORDER BY SALARY;

Data Normalization [and its usages]

- Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies.
- Normalization rules divides larger tables into smaller tables and links them using relationships.
- The purpose of Normalisation in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically.

Here is a list of Normal Forms in SQL:



Data Normalization [and its usages]

- ☐ 1NF (First Normal Form)
- ☐ 2NF (Second Normal Form)
- ☐ 3NF (Third Normal Form)
- ☐ BCNF (Boyce-Codd Normal Form)
- ☐ 4NF (Fourth Normal Form)
- ☐ 5NF (Fifth Normal Form)
- ☐ 6NF (Sixth Normal Form)

Data Normalization [and case study]

Let's understand Normalization database with normalization example with solution:

FULL NAMES	PHYSICAL ADDRESS	MOVIES RENTED	SALUTATION
Janet Jones	First Street Plot No 4	Pirates of the Caribbean, Clash of the Titans	Ms.
Robert Phil	3 rd Street 34	Forgetting Sarah Marshal, Daddy's Little Girls	Mr.
Robert Phil	5 th Avenue	Clash of the Titans	Mr.

Data Normalization [and normal forms]--- 1NF

1NF (First Normal Form) Rules

- Each table cell should contain a single value.
- Each record needs to be unique.

Usage:

Primary Key, Composite Key

Data Normalization [and normal forms]--- 1NF

FULL NAMES	PHYSICAL ADDRESS	MOVIES RENTED	SALUTATION
Janet Jones	First Street Plot No 4	Pirates of the Caribbean	Ms.
Janet Jones	First Street Plot No 4	Clash of the Titans	Ms.
Robert Phil	3 rd Street 34	Forgetting Sarah Marshal	Mr.
Robert Phil	3 rd Street 34	Daddy's Little Girls	Mr.
Robert Phil	5 th Avenue	Clash of the Titans	Mr.

Data Normalization [and normal forms]---- 2NF

2NF (Second Normal Form) Rules

Rule 1- Be in 1NF

Rule 2- Single Column Primary Key that does not functionally dependent on any subset of candidate key

relation

Data Normalization [and normal forms] – 2NF

 Foreign Key

MEMBERSHIP ID	MOVIES RENTED
1	Pirates of the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans

Foreign Key references Primary Key
Foreign Key can only have values present in primary key
It could have a name other than that of Primary Key

 Primary Key

MEMBERSHIP ID	FULL NAMES	PHYSICAL ADDRESS	SALUTATION
1	Janet Jones	First Street Plot No 4	Ms.
2	Robert Phil	3 rd Street 34	Mr.
3	Robert Phil	5 th Avenue	Mr.

Data Normalization [and normal forms] ---- 3NF

3NF (Third Normal Form) Rules

- Rule 1- Be in 2NF
- Rule 2- Has no transitive functional dependencies

We have again divided our tables and created a new table which stores Salutations.

- There are no transitive functional dependencies, and hence our table is in 3NF
- In Table 3 Salutation ID is primary key, and in Table 1 Salutation ID is foreign to primary key in Table

Data Normalization [and normal forms] ---- 3NF

MEMBERSHIP ID	FULL NAMES	PHYSICAL ADDRESS	SALUTATION ID
1	Janet Jones	First Street Plot No 4	2
2	Robert Phil	3 rd Street 34	1
3	Robert Phil	5 th Avenue	1

MEMBERSHIP ID	MOVIES RENTED
1	Pirates of the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans

SALUTATION ID	SALUTATION
1	Mr.
2	Ms.
3	Mrs.
4	Dr.

Data Normalization [and normal forms]

BCNF (Boyce-Codd Normal Form) Rules

- Even when a database is in 3rd Normal Form, still there would be anomalies resulted if it has more than one Candidate Key.
- Sometimes BCNF is also referred as 3.5 Normal Form.

4NF (Fourth Normal Form) Rules

- If no database table instance contains two or more, independent and multivalued data describing the relevant entity, then it is in 4th Normal Form.

Data Normalization [and normal forms]

5NF (Fifth Normal Form) Rules

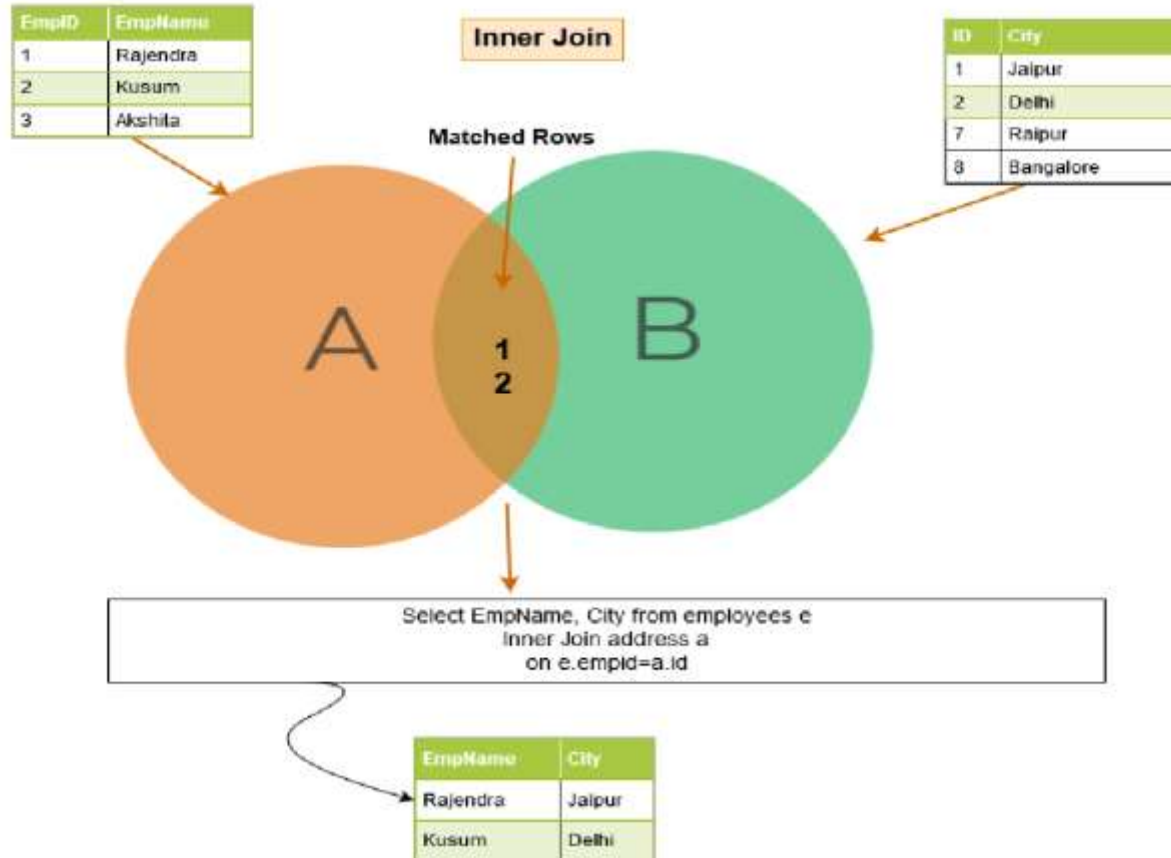
- A table is in 5th Normal Form only if it is in 4NF and it cannot be decomposed into any number of smaller tables without loss of data.

6NF (Sixth Normal Form) Proposed

SQL JOINS [different types and its usages]

- SQL JOINS [different types and its usages] SQL JOIN is a clause that is used to combine multiple tables and retrieve data based on a common field in relational databases.
- SQL JOIN generates meaningful data by combining multiple relational tables.

SQL Inner Join



```
Select EmpName, City  
from employees e  
join address a  
on e.empid=a.id
```

198 %

Results Messages

	EmpName	City
1	Rajendra	Jaipur
2	Kusum	Delhi

SQL Self Join

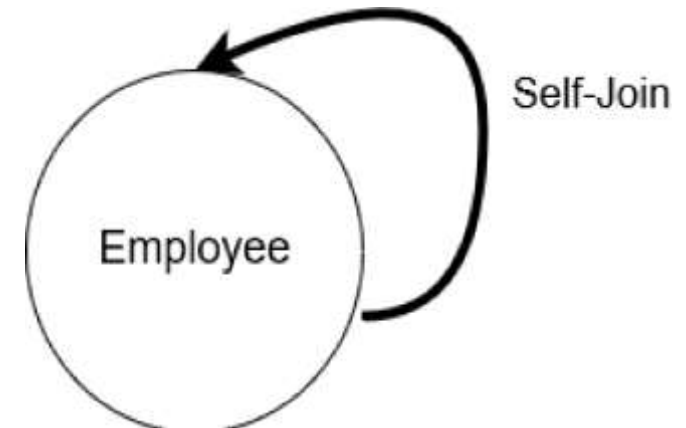
EmpID	Name	EmpMgrid
1	Rajendra	1
2	Mohan	1
3	Amit	1
4	Manoj	1
5	Manish	2
6	Kapil	2



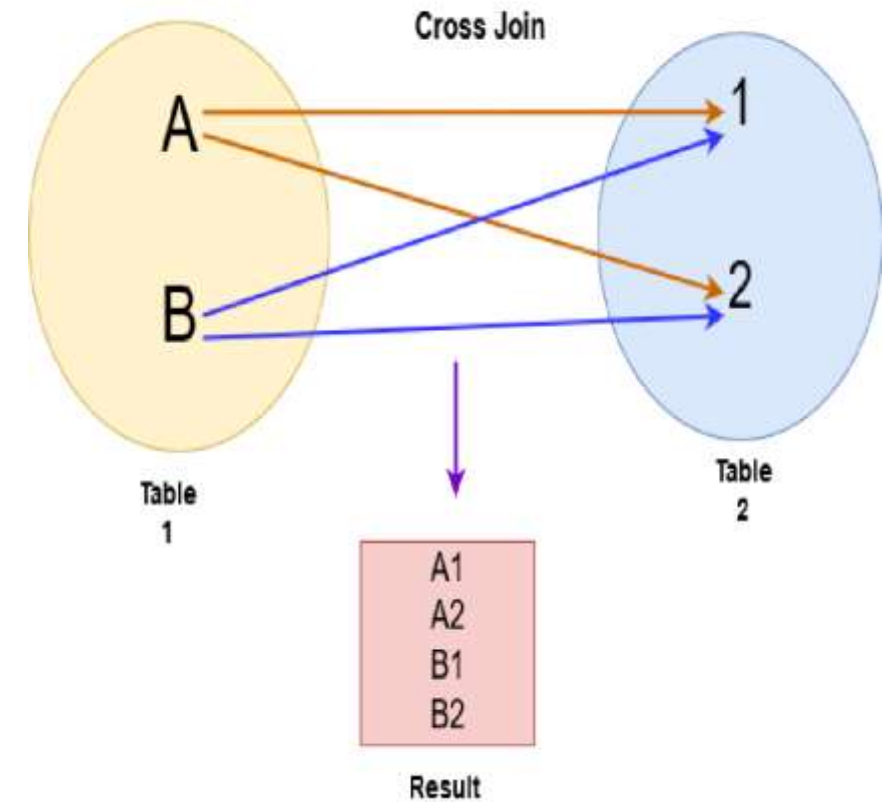
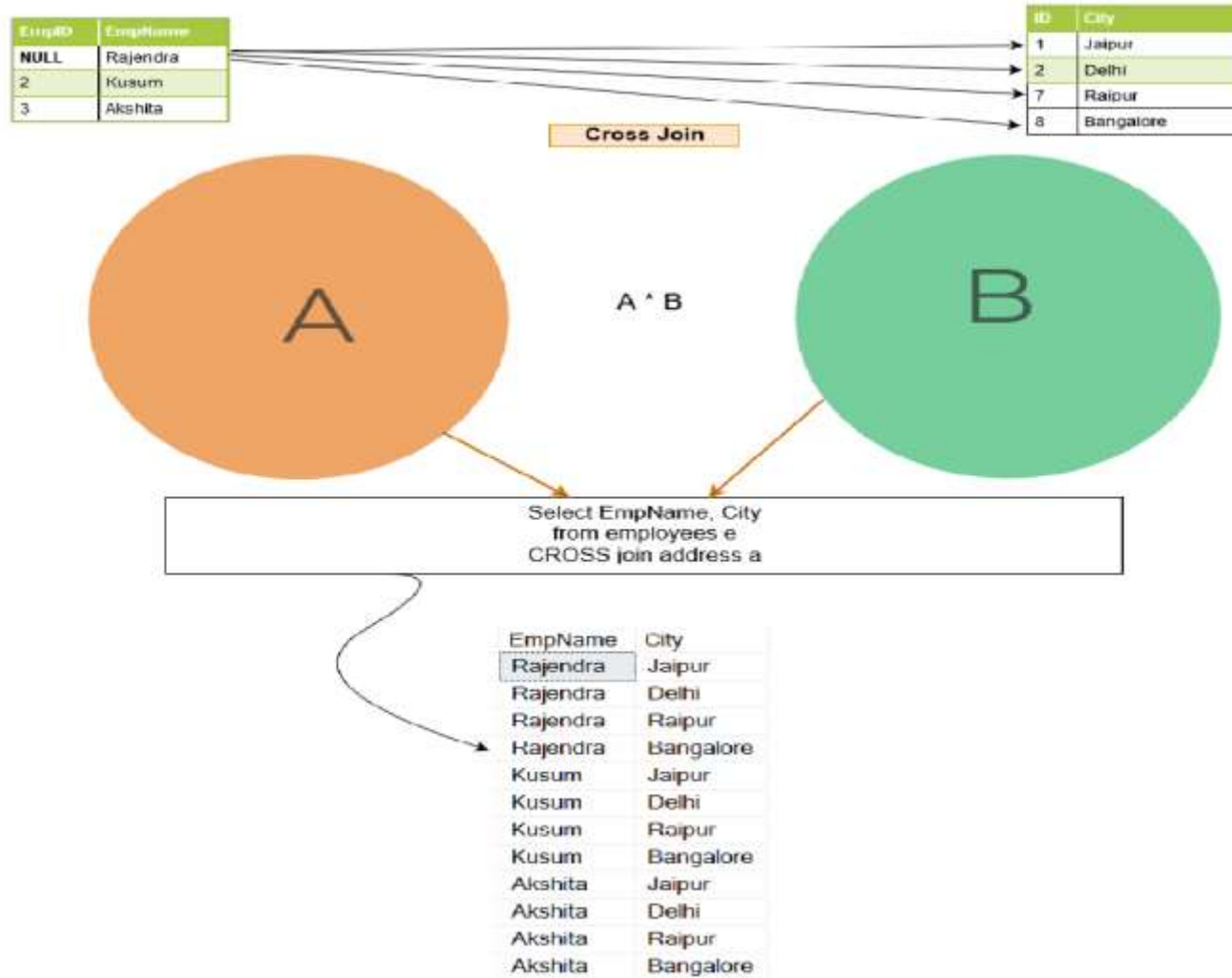
```
SELECT e.EmpID, e.Name, m.Name as Manager FROM Emp e Inner Join Emp m  
On e.EmpMgrid=m.EmpID;
```

```
SELECT e.EmpID, e.Name,  
m.Name as Manager  
FROM Emp e Inner Join Emp m  
On e.EmpMgrid=m.EmpID;
```

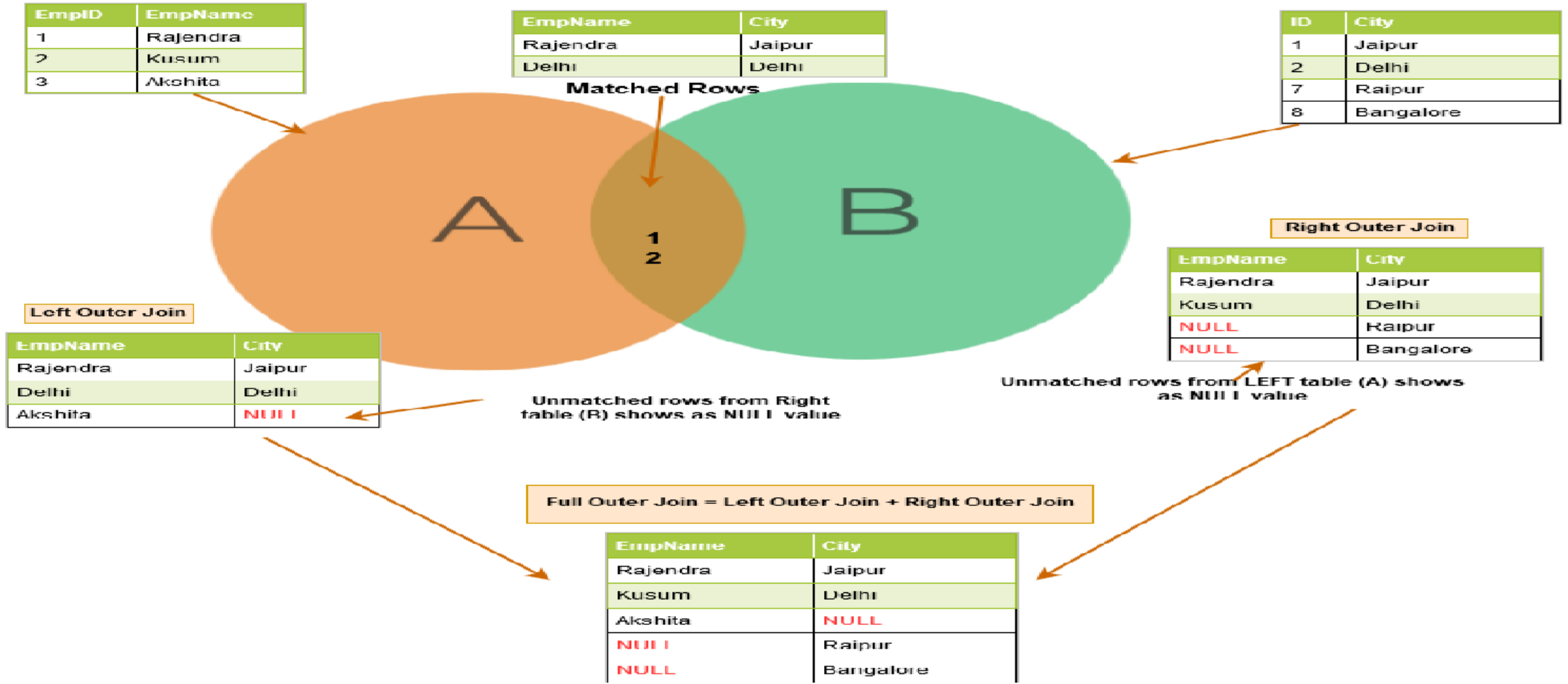
EmpID	Name	Manager
1	Rajendra	Rajendra
2	Mohan	Rajendra
3	Amit	Rajendra
4	Manoj	Rajendra
5	Manish	Mohan
6	Kapil	Mohan



SQL Cross Join



SQL Outer Join

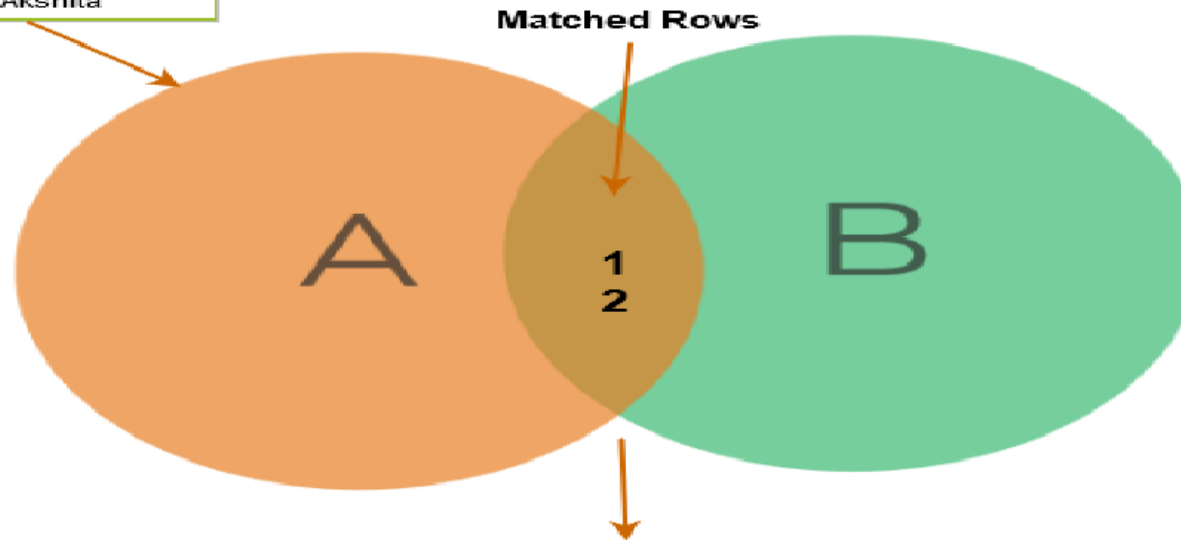


SQL Left Outer Join

EmpID	EmpName
1	Rajendra
2	Kusum
3	Akshita

Left Outer Join

ID	City
1	Jaipur
2	Delhi
7	Raipur
8	Bangalore



Select EmpName, City from employees e
Left Outer Join address a
on e.empid=a.id

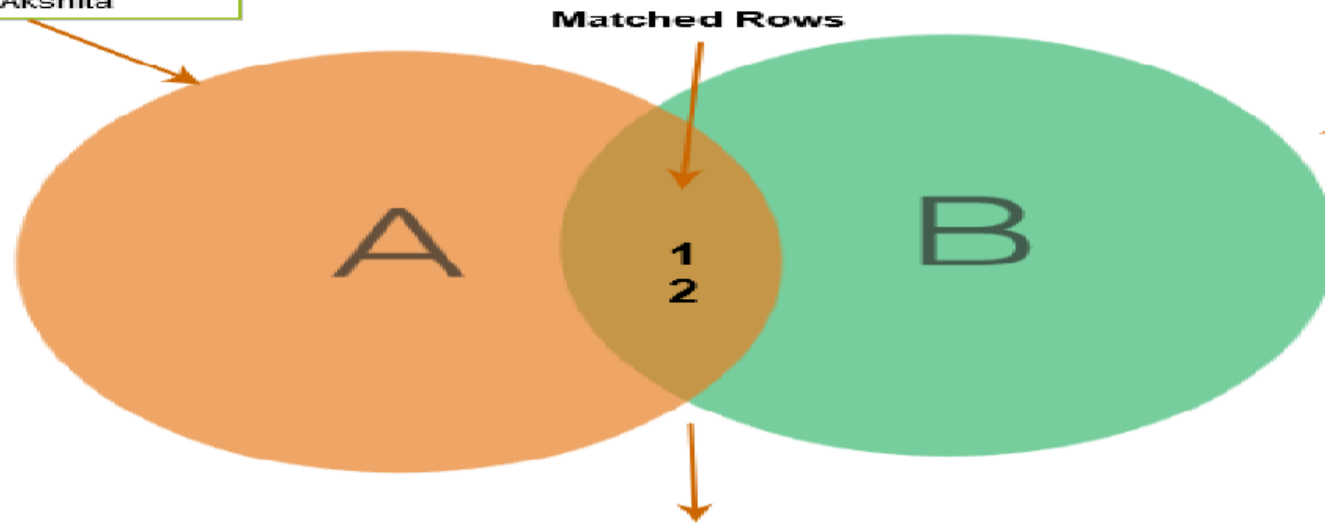
EmpName	City
Rajendra	Jaipur
Kusum	Delhi
Akshita	NULL

SQL Right Outer Join

EmpID	EmpName
1	Rajendra
2	Kusum
3	Akshita

Right Outer Join

ID	City
1	Jaipur
2	Delhi
7	Raipur
8	Bangalore



Select EmpName, City from employees e
Right Outer Join address a
on e.empid=a.id

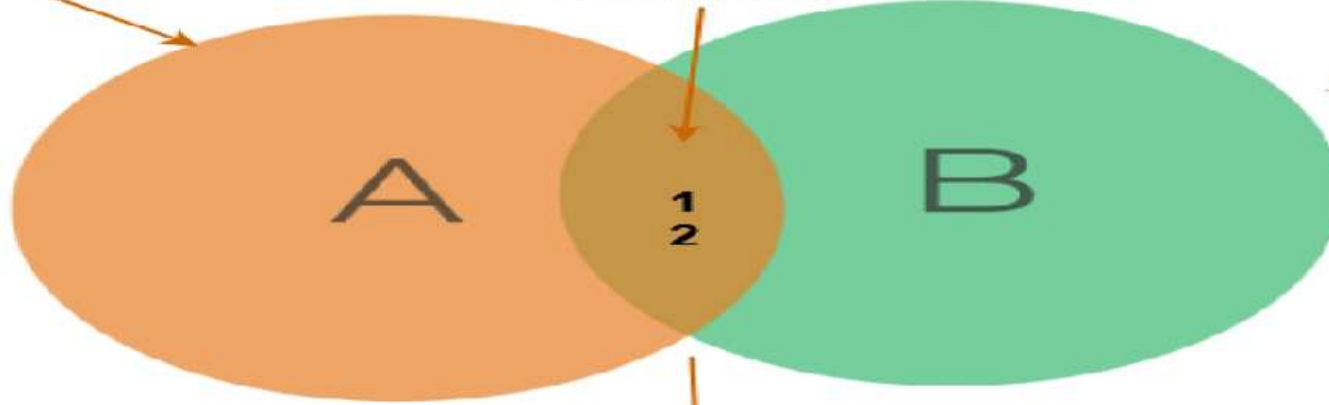
EmpName	City
Rajendra	Jaipur
Kusum	Delhi
NULL	Raipur
NULL	Bangalore

SQL Full Outer Join

EmpID	EmpName
1	Rajendra
2	Kusum
3	Akshita

Full Outer Join

Matched Rows

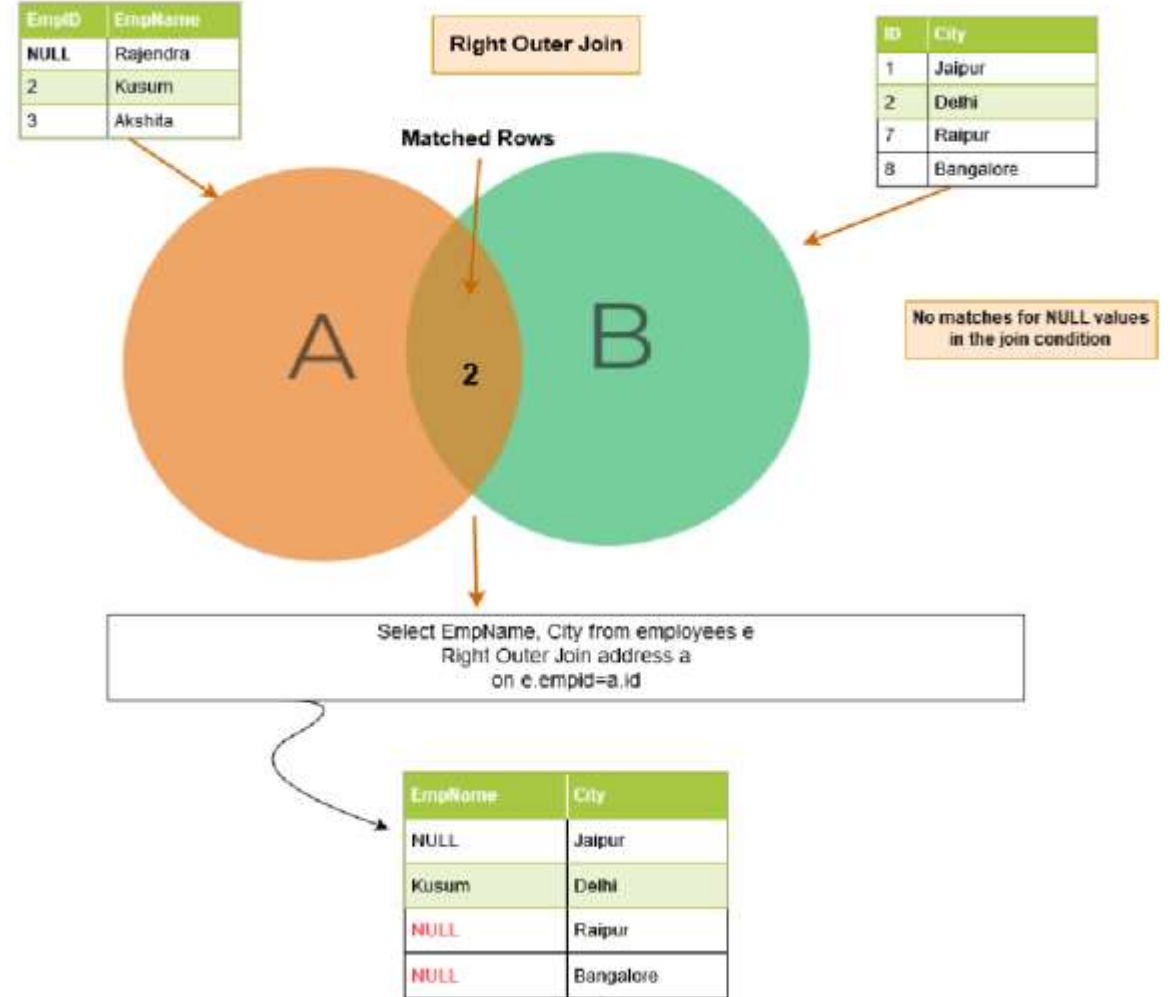
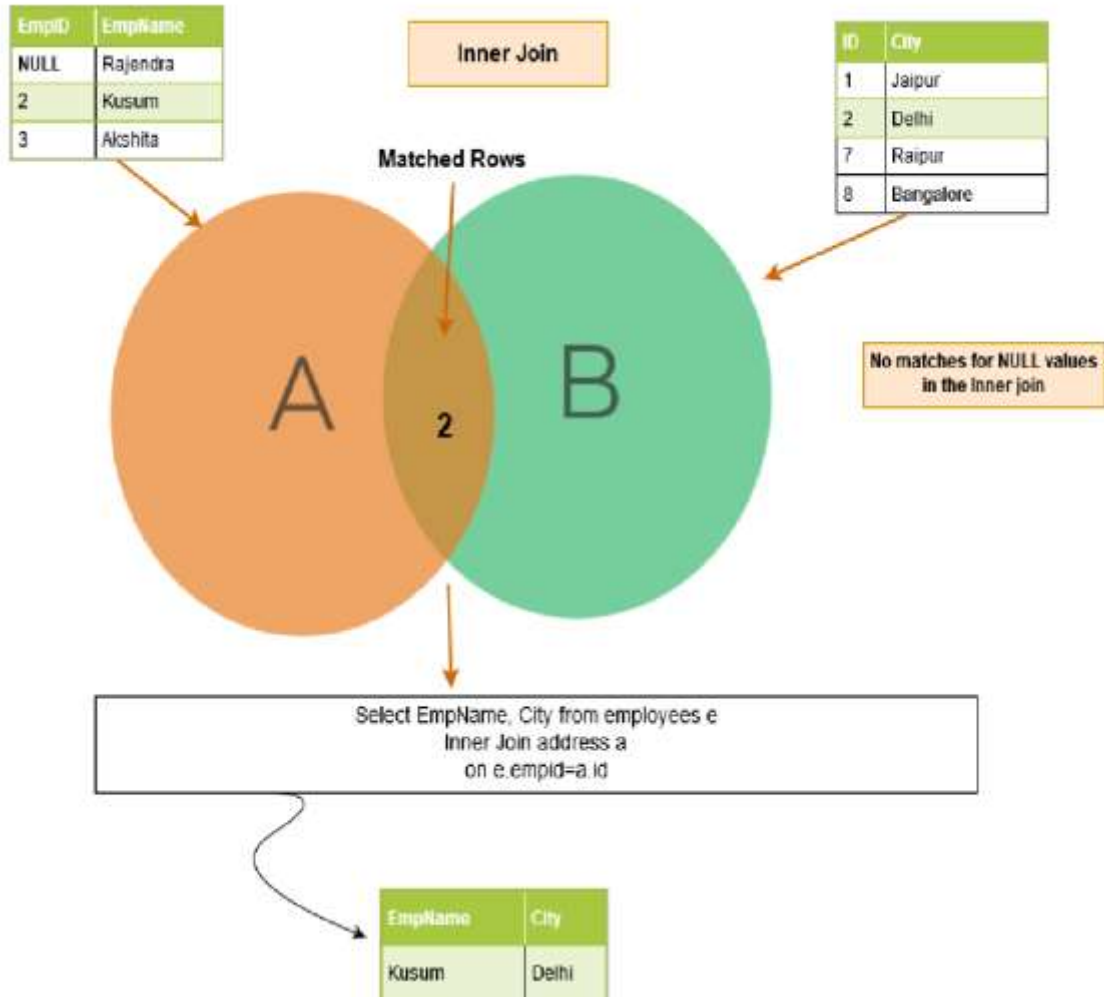


ID	City
1	Jaipur
2	Delhi
7	Raipur
8	Bangalore

Select EmpName, City from employees e
Full Outer Join address a
on e.empid=a.id

EmpName	City
Rajendra	Jaipur
Kusum	Delhi
Akshita	NULL
NULL	Raipur
NULL	Bangalore

Handling NULL Values [in SQL JOINS]



Best Practices [in SQL JOINS]

Here are a few important best practices to remember and apply when using SQL joins:

- ☐ You should always use a table alias while using the joins in the queries.
- ☐ Always use the two-part name [table alias].[column] name format for columns in queries.

Best Practices [in SQL JOINS]

- ❑ In the case of multiple SQL joins in a query, you should use the logical orders of the tables in such a way to satisfy your data requirement and minimize the data flow between various operators of the execution plan.
- ❑ You can combine multiple joins such as inner join, outer join and self-join together. However, you should use the joins and their orders to get the required data

Database Encryption and Decryption

- Database encryption is the process of converting data, within a database, in plain text format into a meaningless cipher text by means of a suitable algorithm.
- Database decryption is converting the meaningless cipher text into the original information using keys generated by the encryption algorithms.

Database Encryption and Decryption

The data encryption methods

Most internet security (IS) professionals break down encryption into three distinct methods:

- **Symmetric**
- **Asymmetric**
- **Hashing**

Database Encryption and Decryption

Hashing

- In terms of security, hashing is a technique used to encrypt data and generate unpredictable hash values.
- It is the hash function that generates the hash code, which helps to protect the security of transmission from unauthorized users.

Hashing algorithm

- provides a way to verify that the message received is the same as the message sent.
- It can take a plain text message as input and then computes a value based on that message.

Database Encryption and Decryption

MD5

- MD5 is not a reversible encryption algorithm. It's a [one-way hash](#).
- The aim of a hash is to allow you to verify (with some degree of uncertainty) whether
someone had the same original information as was used to create the hash.
- It does *not* allow you to get back from the hash to the original information.

Database Encryption and Decryption

```
INSERT INTO tbllogin(User_type_id, username, password) VALUES(2, 'rahulRam',  
MD5('rahul123Ram'));
```

```
SELECT * FROM tbllogin WHERE password = MD5('rahul123Ram') AND username =  
'rahulRam'
```

Stored Procedure

A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again.

So if you have an SQL query that you write over and over again, save it as a stored procedure, and then just call it to execute it.

You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed.

Stored Procedure Syntax

```
CREATE PROCEDURE procedure_name  
AS  
sql_statement  
GO;
```

Execute a Stored Procedure

```
EXEC procedure_name;
```


SQL Views

View is a virtual table based on the result-set of an SQL statement.

A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

You can add SQL statements and functions to a view and present the data as if the data were coming from one single table.

A view is created with the CREATE VIEW statement.

CREATE VIEW Syntax

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

THANK YOU